

SAVCNG ExcelDNA - Documentación Técnica Completa

Versión: 2.2.4.0

Última Actualización: 20 de junio de 2026

Estado: En Desarrollo Continuo

Tabla de Contenidos

1. [Descripción General](#)
 2. [Arquitectura del Proyecto](#)
 3. [Componentes Principales](#)
 4. [Sistema de Validaciones Detallado](#)
 - [4.1 Validación de Decimales](#)
 - [4.2 Validación de Catálogos](#)
 - [4.3 Validación NS](#)
 - [4.4 Validación de Formato Texto](#)
 - [4.5 Validación de Bloqueo Dinámico Mejorado](#)
 - [4.6 Validación de Campos Vacíos Inteligente](#)
 5. [FrmValidaciones.cs - Análisis Profundo](#)
 6. [Flujo de Trabajo](#)
 7. [Especificaciones Técnicas](#)
 8. [Patrones de Código y Mejores Prácticas](#)
 9. [Troubleshooting y Casos Especiales](#)
-

Descripción General

SAVCNG ExcelDNA es un complemento (Add-in) de Microsoft Excel desarrollado en C# (.NET Framework 4.8) que proporciona un sistema avanzado de validación de datos para archivos de censo de gobierno.

El proyecto utiliza **ExcelDNA** como framework de integración con Excel, permitiendo crear un menú personalizado en la cinta de opciones (Ribbon) de Excel desde el cual se pueden cargar archivos de censo y aplicar múltiples validaciones de integridad de datos.

Propósito Principal

- Garantizar la calidad e integridad de los datos en archivos de censo
- Proporcionar una interfaz gráfica intuitiva para validación de datos
- Detectar y resaltar errores de formato y contenido
- Facilitar listas desplegadas (Data Validation) personalizadas
- Validar presencia de respuestas "No Aplica" (NS) con alertas contextuales
- Validar formato de texto: mayúsculas, sin espacios extra, caracteres válidos
- Crear reglas de bloqueo dinámico condicionadas a valores de otras celdas
- Soporte automático para matrices paralelas y evaluaciones fila por fila

- Mensajes de alerta personalizados que aparecen dinámicamente
- Detectar y resaltar campos vacíos en matrices con validación inteligente de obligatoriedad
- Coexistencia automática entre múltiples validaciones sin conflictos

Tecnologías Utilizadas

- **Lenguaje:** C# 7.3
- **Framework:** .NET Framework 4.8
- **Librería de Interop:** Microsoft.Office.Interop.Excel
- **Plugin Framework:** ExcelDNA
- **UI:** Windows Forms (.NET Framework)

Arquitectura del Proyecto

```
SAVCNG_ExcelDNA/  
├─ MenuCenso.cs           (Punto de entrada y menú)  
├─ FrmValidaciones.cs     (Lógica principal de validaciones)  
├─ FrmValidaciones.Designer.cs (Diseño de formulario)  
├─ Properties/  
│   └─ AssemblyInfo.cs    (Metadatos del ensamblado)  
├─ bin/  
├─ obj/  
└─ documentacion/  
    └─ doc_savcng_YYYY_MM_DD.md (Documentación técnica)
```

Patrón Arquitectónico

- **Model:** Las colecciones (List, Dictionary) almacenan datos de validación y rangos
- **View:** Windows Forms (FrmValidaciones) para la interfaz de usuario
- **Controller:** Métodos de eventos que procesan interacciones del usuario
- **Service:** Métodos estáticos y funciones auxiliares para lógica compartida

Componentes Principales

1. **MenuCenso.cs** - Punto de Entrada

Responsable de la integración con Excel y la interfaz de usuario del ribbon.

Características Principales:

```
public class MenuCenso : ExcelRibbon  
{  
    // Define un menú personalizado en la cinta de Excel
```

```
public override string GetCustomUI(string RibbonID)
{
}
```

Funcionalidades:

- **XML Ribbon:** Define la estructura del menú en la pestaña "SAVCNG - Censo"
 - **Diálogo de Archivo:** Permite seleccionar archivos de censo (.xlsx, .xlsm, .xlsb)
 - **Gestión de Workbook:** Abre el archivo y lo pasa a la ventana de validaciones
 - **Manejo de Errores:** Captura excepciones durante la carga de archivos
-

2. FrmValidaciones.cs - Motor de Validaciones

Este es el corazón del aplicativo. Implementa todas las validaciones de datos y maneja la interacción del usuario.

Variables de Estado Principales

```
private Excel.Workbook _libroCenso;
private Excel.Range _rangoCapturado;
private Excel.Range _rangoCatalogo;
private System.Collections.Generic.List<Excel.Range> _pendientesCatalogo;
private System.Collections.Generic.Dictionary<string, string> _preguntaPorRango;
```

Sistema de Validaciones Detallado

4.1 Validación de Decimales

Objetivo: Detectar y resaltar celdas que contienen números con decimales cuando solo se permiten números enteros.

Tipo: Formato Condicional (Visual)

Restricción: No (solo resalta)

Motor: `ESNUMERO()` + `TRUNCAR()`

Color: Fondo amarillo, texto rojo, negrita

4.2 Validación de Catálogos

Objetivo: Crear listas desplegables (Data Validation) con opciones predefinidas.

Tipo: Data Validation - Lista

Restricción: Sí (solo permite valores de la lista)

Motor: Rango de opciones o entrada manual

Características:

- Entrada manual de opciones (ej: "1,2,9")
 - Seleccionar celdas con múltiples opciones de extracción
 - Extracción SOLO NÚMEROS o TEXTO EXACTO
 - Uso de rangos como referencias directas
 - Limpieza automática de caracteres innecesarios
-

4.3 Validación NS (No Aplica)

Objetivo: Permitir números ≥ 0 o valor "NS", con alerta automática cuando se usa NS.

Tipo: Data Validation - Custom + Formato + Fórmula

Restricción: Sí (solo permite números ≥ 0 o "NS")

Motor: `O(Y(ESNUMERO,  $\geq 0$ ), "NS")`

Características:

- Validación restrictiva con mensaje personalizado
 - Alerta automática en fila siguiente cuando hay registros NS
 - Mensaje combinado y formateado (Arial 9pt, Negrita, Color Dorado)
 - Manejo de rangos no contiguos (Areas)
-

4.4 Validación de Formato Texto

Objetivo: Garantizar que el texto cumpla formato específico: mayúsculas, sin espacios extra.

Tipo: Data Validation - Custom

Restricción: Sí (previene entrada inválida)

Motor: `IGUAL() + ESPACIOS()`

Reglas Implementadas:

1. ✓ Solo mayúsculas (A-Z, Ñ)
2. ✓ Espacios simples (sin dobles espacios)
3. ✓ Sin espacios al inicio o final

Fórmula:

```
=Y(IGUAL(A1;MAYUSC(A1));LARGO(A1)=LARGO(ESPACIOS(A1)))
```

4.5 Validación de Bloqueo Dinámico Mejorado ★ (v2.2.4)

Objetivo: Crear reglas de bloqueo condicional basadas en valores de otras celdas con soporte para celdas vacías.

Tipo: Data Validation - Custom + Formatos Condicionales

Restricción: Sí (bloquea entrada si condición no se cumple)

Motor: `CONTAR.SI()` con regla de celda vacía

Nuevas Características v2.2.4:

1. Regla de Celda Vacía (Paso 3.5):

- Pregunta: ¿Desbloquear si la condición está vacía?
- Sí: Se puede escribir si está en blanco
- NO: Estricto (se bloquea si está vacío)

2. Dos Formatos Dinámicos:

- **Gris Bloqueado:** Cuando la condición no se cumple
- **Azul Obligatorio:** Cuando se desbloquea pero está vacío (opcional)

3. Inteligencia de Jerarquía:

- `StopIfTrue = true` en formato gris previene reaplicación
- `StopIfTrue = true` en formato azul evita superponer
- Conserva fondo blanco para limpiar formatos previos

4. Auditoría de Coexistencia:

```
if (_rangoCapturado.FormatConditions.Count > 0)
{
    // Pregunta si conservar o eliminar formatos previos
    // Permite apilar bloqueos con validación de blancos
}
```

5. Fórmulas Dinámicas según Configuración:

Si celda vacía = Sí:

Validación: `=O(ESBLANCO(Condición);CONTAR.SI(Condición;Operador Valor)>0)`
Sombreado: `=Y(ESBLANCO(Condición)=FALSO;CONTAR.SI(Condición;Operador Valor)=0)`

Si celda vacía = No:

Validación: `=CONTAR.SI(Condición;Operador Valor)>0`
Sombreado: `=CONTAR.SI(Condición;Operador Valor)=0`

Flujo Mejorado de 4.5 Pasos:

1. **Paso 1:** Seleccionar rango de condición
2. **Paso 2:** Elegir operador (=, <>, >, <, >=, <=)
3. **Paso 3:** Ingresar valor del criterio
4. **Paso 3.5:** ★ NUEVO - Elegir comportamiento con celdas vacías
5. **Paso 4:** Opcional - Resalte azul si desbloqueado pero vacío

Contexto de Uso:

Ejemplo: Campo "Especifique" que solo se activa si P1 = "Otra"

Si celda vacía = Sí:

- └ Si P1 = "Otra" y "Especifique" vacío → Azul (obligatorio)
- └ Si P1 = "Otra" y "Especifique" completo → Normal
- └ Si P1 = "" (vacío) → Desbloqueado igualmente
- └ Si P1 ≠ "Otra" → Gris (bloqueado)

Si celda vacía = No:

- └ Si P1 = "Otra" y "Especifique" vacío → Azul (obligatorio)
- └ Si P1 = "Otra" y "Especifique" completo → Normal
- └ Si P1 = "" (vacío) → Gris (bloqueado)
- └ Si P1 ≠ "Otra" → Gris (bloqueado)

4.6 Validación de Campos Vacíos Inteligente ★ (v2.2.4)

Objetivo: Detectar y resaltar campos vacíos en matrices donde la obligatoriedad depende de si la fila tiene datos en la columna base.

Tipo: Data Validation - Fórmula Matricial + Formato Condicional

Restricción: No (solo resalta)

Motor: SUMPRODUCT() con álgebra booleana

Novedad v2.2.4:

- Motor matricial dinámico 1D x 1D mejorado
- Coexistencia inteligente con validación de Bloqueo
- Álgebra booleana sin AND/OR para máxima compatibilidad
- Auditoría de formatos previos para no sobrescribir

Contexto: En censos matriciales, cuando el usuario completa la columna base, todas las columnas en esa fila se vuelven obligatorias.

Ejemplo Real:

Columna Base (D)	Matriz de Captura (M:S)
Centro Municipal	[Amarillo: Obligatorios]
Colegio Privado	[Amarillo: Obligatorios]
[Vacío]	[Sin color: Opcionales]
Hospital Regional	[Amarillo: Obligatorios]

Flujo de Configuración v2.2.4 (2 Pasos):

Paso 1: Ubicación de Alerta

```
// Usuario selecciona celda donde aparecer á el mensaje en AZUL
// Sistema automáticamente combina si hay múltiples celdas
Excel.Range rangoDestino = (Excel.Range)excelApp.InputBox(..., 8);

if (rangoDestino.Count > 1) { rangoDestino.Merge(); }
```

Paso 2: Mensaje de Alerta

```
// Usuario escribe el texto del mensaje
// Sistema lo inyecta con .Formula (INGLÉS universal)
string textoAlerta = excelApp.InputBox(..., 2);

string formulaGlobal = $"=IF(SUMPRODUCT(...), \"{textoAlerta}\", \"\")";
rangoDestino.Formula = formulaGlobal;
```

Motor Matricial Dinámico (Fórmula de Alerta):

```
// Construir lista de activadores: (Col1<>"") + (Col2<>"") + ...
System.Collections.Generic.List<string> listaActivadores = new();
for (int i = 1; i <= _rangoCapturado.Columns.Count; i++)
{
    Excel.Range col = (Excel.Range)_rangoCapturado.Columns[i];
    string dirColAbs = col.get_Address(true, true, Excel.XlReferenceStyle.xlA1, false);
    listaActivadores.Add($"({dirColAbs}<>"")");
}

// Construir lista de vacíos: (Col1="") + (Col2="") + ...
System.Collections.Generic.List<string> listaVacios = new();
for (int i = 1; i <= _rangoCapturado.Columns.Count; i++)
{
    Excel.Range col = (Excel.Range)_rangoCapturado.Columns[i];
```

```

        string dirColAbs = col.get_Address(true, true, Excel.XlReferenceStyle.xlA1,
false);
        listaVacios.Add($"({dirColAbs}=\"\")");
    }

    // Unir: suma de activadores * suma de vacíos
    string motorActivadores = string.Join("+", listaActivadores);
    string motorVacios = string.Join("+", listaVacios);

    // Fórmula final: Si (activadores > 0 Y vacíos > 0) → mostrar mensaje
    string formulaGlobal = $"=IF(SUMPRODUCT(({motorActivadores})*({motorVacios}))>0, \"{
textoAlerta}\", \"\")";

```

Formato Condicional (Álgebra Booleana Fila por Fila):

```

// Obtener fila inicial
int filaInicial = _rangoCapturado.Row;

// Construir suma de validaciones por fila: ($M52<>"") + ($N52<>"") + ...
System.Collections.Generic.List<string> validacionFila = new();
for (int c = 1; c <= _rangoCapturado.Columns.Count; c++)
{
    Excel.Range celdaIteracion = (Excel.Range)_rangoCapturado.Cells[1, c];
    string colLetra = celdaIteracion.Address.Split('$')[1];

    validacionFila.Add($"({colLetra}{filaInicial}<>\"\")");
}

// Unir: suma de activadores por fila
string sumaFila = string.Join("+", validacionFila);

// Fórmula: Si (esta celda está vacía) Y (la fila tiene datos) → pintar azul
string formulaCondicionalMatematica = $"=({celdaCapRelativa}=\"\")*
(({sumaFila})>0)";

Excel.FormatCondition formatoAmarillo =
(Excel.FormatCondition)_rangoCapturado.FormatConditions.Add(
    Excel.XlFormatConditionType.xlExpression, Type.Missing,
    formulaCondicionalMatematica);

formatoAmarillo.Interior.Color =
System.Drawing.ColorTranslator.ToOle(System.Drawing.Color.FromArgb(47, 117, 181));

```

Auditoría de Coexistencia v2.2.4:


```
// Detectar si ya hay formatos previos (ej: Bloqueo Dinámico)
if (_rangoCapturado.FormatConditions.Count > 0)
{
    DialogResult respFormato = MessageBox.Show(
        "Se detectaron reglas de formato condicional previas...\n\n" +
        "¿Deseas CONSERVAR las reglas existentes?\n\n" +
        "SÍ = Conservar (no borrar tus bloques grises).\n" +
        "NO = Eliminar y aplicar solo el formato de blancos.",
        "Formatos Condicionales Detectados",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Warning);

    if (respFormato == DialogResult.No)
    {
        _rangoCapturado.FormatConditions.Delete();
    }
    // Si es SÍ, se CONSERVAN los formatos y se apilan los nuevos
}
else
{
    _rangoCapturado.FormatConditions.Delete();
}
```

Ventajas del Motor Inteligente v2.2.4:

- ✓ **Sin errores #N/D:** Vectores siempre alineados
- ✓ **Eficiencia:** Una fórmula por columna, no por celda
- ✓ **Escalabilidad:** Matrices de cualquier tamaño
- ✓ **Precisión fila-por-fila:** Resalta exactamente lo necesario
- ✓ **Multiidioma:** Usa .Formula (INGLÉS) universal
- ✓ **Coexistencia:** Detecta formatos previos y los conserva
- ✓ **Auditoría:** Pregunta antes de sobrescribir

FrmValidaciones.cs - Análisis Profundo

5.1 Método: btnCapturarRango_Click

Responsabilidad: Capturar el rango seleccionado por el usuario y detectar contexto.

```
if (seleccion is Excel.Range)
{
    _rangoCapturado = (Excel.Range)seleccion;
    string pregunta = ObtenerNumeroPregunta(_rangoCapturado);
    lblPregunta.Text = "Pregunta detectada: " + pregunta;
    lblRangoSeleccionado.Text = "Rango seleccionado: " +
```

```

_rangoCapturado.Address.Replace("$", "");
    MessageBox.Show("Se capturó correctamente el rango: " + _rangoCapturado.Address,
        "Captura exitosa", MessageBoxButtons.OK, MessageBoxIcon.Information);
}

```

5.2 Método: btnAplicar_Click - Estructura General v2.2.4

```

btnAplicar_Click()
|
|─ if (chkDecimales.Checked)
|   └─ Validación de Decimales
|
|─ else if (chkCatalogos.Checked)
|   └─ Validación de Catálogos (Manual o Seleccionar)
|
|─ else if (chkNS.Checked)
|   └─ Validación NS con Alerta
|
|─ else if (chkFormatoTexto.Checked)
|   └─ Validación de Formato Texto
|
|─ else if (chkBloqueo.Checked)
|   └─ Validación de Bloqueo Dinámico (Paso 3.5 NUEVO)
|       └─ Paso 1: Rango de condición
|       └─ Paso 2: Operador
|       └─ Paso 3: Valor
|       └─ Paso 3.5: ★ Celda vacía (SÍ/NO)
|       └─ Paso 4: Resalte azul (opcional)
|       └─ Auditoría: Detectar formatos previos
|       └─ Aplicar: Data Validation + Formatos
|
|─ else if (chkBlancos.Checked)
|   └─ Validación de Campos Vacíos (2 pasos)
|       └─ Paso 1: Ubicación mensaje azul
|       └─ Paso 2: Texto del mensaje
|       └─ Motor: SUMPRODUCT dinámico
|       └─ Auditoría: Coexistencia con Bloqueo
|       └─ Formato: Álgebra booleana fila-por-fila

```

5.3 Método CheckedChanged v2.2.4

```

private void chkBlancos_CheckedChanged(object sender, EventArgs e)
{

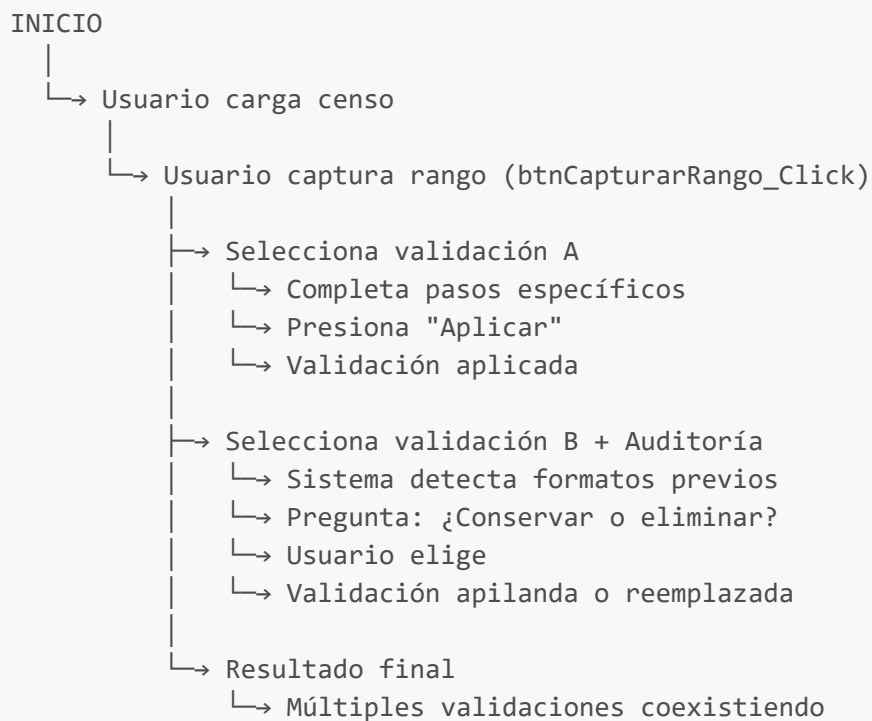
```

```

if (chkBlancos.Checked)
{
    // Validamos la regla de negocio
    if (_libroCenso == null || _rangoCapturado == null)
    {
        MessageBox.Show("Operación denegada: Carga un censo y define el rango
primero.",
                        "Advertencia Arquitectónica", MessageBoxButtons.OK,
        MessageBoxIcon.Exclamation);
        chkBlancos.Checked = false;
    }
}
}

```

Flujo de Trabajo v2.2.4



Especificaciones Técnicas v2.2.4

Requisitos del Sistema

Aspecto	Requisito
Versión .NET	.NET Framework 4.8

Aspecto	Requisito
Versión C#	7.3
Versión Excel	2013 o superior
SO	Windows 7/8/10/11 (32 o 64 bits)

Rendimiento v2.2.4

Operación	Tiempo Estimado
Validación bloqueo (4.5 pasos + auditoría)	10-15 seg
Validación blancos matriz 50x7	5-8 seg
Coexistencia Bloqueo + Blancos	15-20 seg
Memoria usada	50-150 MB

Límites Conocidos v2.2.4

- **Máximo columnas en matriz:** ~50 (límite práctico Excel)
- **Máximo filas en matriz:** ~1000 (por rendimiento)
- **Máximo texto mensaje:** 255 caracteres
- **Máximo formatos condicionales:** 3 reglas simultáneas recomendadas

Patrones de Código v2.2.4

Pattern 1: Auditoría de Coexistencia

```
// CORRECTO: Detectar y preguntar antes de sobrescribir
if (_rangoCapturado.FormatConditions.Count > 0)
{
    DialogResult respFormato = MessageBox.Show(
        "¿Deseas CONSERVAR formatos previos?",
        "Auditoría", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);

    if (respFormato == DialogResult.No)
    {
        _rangoCapturado.FormatConditions.Delete();
    }
}
```

Pattern 2: Construcción Dinámica de Listas

```
// CORRECTO: Iterar columnas para construir fórmulas
System.Collections.Generic.List<string> lista = new();
for (int i = 1; i <= _rangoCapturado.Columns.Count; i++)
{
    Excel.Range col = (Excel.Range)_rangoCapturado.Columns[i];
    string dirColAbs = col.get_Address(true, true, Excel.XlReferenceStyle.xlA1,
false);
    lista.Add($"({dirColAbs}<>\\"");
}
string resultado = string.Join("+", lista);
```

Pattern 3: Fórmulas Condicionales por Parámetro

```
// CORRECTO: Bifurcación de fórmulas según entrada
string formulaValidacion = "";
if (respuestaBlanco == DialogResult.Yes)
{
    formulaValidacion = $"=0(ESBLANCO({dir}){separador}CONTAR.SI({dir}{separador}
{criterio})>0)";
}
else
{
    formulaValidacion = $"=CONTAR.SI({dir}{separador}{criterio})>0";
}
```

Troubleshooting v2.2.4

Problema 1: Formatos previos se pierden

Causa: No conservar formatos existentes al aplicar nueva validación

Solución:

```
// Sistema pregunta automáticamente si formatos > 0
// Responder SÍ para conservar
```

Problema 2: Fórmula muestra #N/D

Causa: Vectores de tamaño diferente en SUMPRODUCT

Solución: Sistema itera columna-por-columna automáticamente (sin intervención)

Problema 3: Bloqueo y Blancos compiten

Causa: No detectar coexistencia entre validaciones

Solución: Auditoría automática en ambas validaciones (v2.2.4)

Resumen de Cambios v2.2.4

🌟 Nuevas Características

1. Paso 3.5 en Bloqueo Dinámico ⭐

- Preguntar si desbloquear con celda vacía
- Fórmulas dinámicas según respuesta
- Mejor control del comportamiento

2. Auditoría de Coexistencia ⭐

- Detectar formatos previos
- Preguntar antes de sobrescribir
- Permitir apilar validaciones

3. Validación de Blancos Mejorada ⭐

- Solo 2 pasos (antes 3)
- Auditoría automática
- Coexistencia con Bloqueo

📊 Matriz de Validaciones Disponibles (v2.2.4)

Validación	Tipo	Restricción	Auditoría	Versión
Decimales	FormatCondition	No	No	v1.0
Catálogos	Data Validation (List)	Sí	No	v2.2.2
NS	Data Validation (Custom)	Sí	No	v2.0
Formato Texto	Data Validation (Custom)	Sí	No	v2.1
Bloqueo Dinámico	Data Validation + Format	Sí	Sí ⭐	v2.2.4
Campos Vacíos	Format + Fórmula	No	Sí ⭐	v2.2.4

Conclusión v2.2.4

SAVCNG ExcelDNA versión 2.2.4 introduce auditoría automática de coexistencia entre validaciones. La nueva **Auditoría de Formatos Condicionales** permite apilar múltiples validaciones sin conflictos, representando un avance significativo hacia la robustez y flexibilidad del sistema.

Fortalezas de v2.2.4: ✓ Seis tipos de validación complementarios ✓ Auditoría automática de coexistencia ✓ Formatos dinámicos según configuración ✓ Interfaz guiada y accesible ✓ Máxima compatibilidad multiidioma ✓ Soporte para matrices paralelas complejas

Estado: Listo para producción

Rama Git: Desarrollo

Próxima Versión: 2.2.5 (optimizaciones de rendimiento)

Versión: 2.2.4.0

Última Actualización: 20 de junio de 2026

Estado: En Desarrollo Continuo